

SecureCodeOps AI

Security Assessment Report: SecureCodeOps AI Demo Project (Vulnerable App)

Multi-Agent Automated DevSecOps Audit • Generated: 2026-09-06 16:08:48 UTC

1. Executive Summary

Security analysis for repository 'SecureCodeOps AI Demo Project (Vulnerable App)' completed. The overall Security Score is 0.0/100. Analysis identified 9 total security findings (5 Critical, 3 High, 1 Medium, 0 Low), 7 STRIDE architectural threats, 18.0% compliance posture across major frameworks, and 11 tracked package dependencies with 7 known CVE advisories. 4 context-aware patches were synthesized and verified with sandbox re-scans.

Overall Security Score 0.0/100	Compliance Score 18.0%	Total Vulnerabilities 9	STRIDE Threats 7
-----------------------------------	---------------------------	----------------------------	---------------------

2. Critical & High Security Findings

Severity	Title	CWE / OWASP	Location	Scanner
CRITICAL	SQL Injection via Dynamic String Formatting / Concatenation	CWE-89 A03:2021-Injection	app.py:49	semgrep
CRITICAL	Command Injection via Shell Subprocess	CWE-78 A03:2021-Injection	app.py:70	semgrep
CRITICAL	Hardcoded Secret: AWS Access Key ID	CWE-798 A02:2021-Cryptographic Failures	src/userController.js:6	gitleaks
CRITICAL	Vulnerable Dependency: jsonwebtoken@8.5.1	CWE-1395 A06:2021-Vulnerable and Outdated Components	package.json:1	trivy
CRITICAL	Vulnerable Dependency: pyyaml@5.3.1	CWE-1395 A06:2021-Vulnerable and Outdated Components	requirements.txt:1	trivy
HIGH	Insecure Deserialization via Pickle/Yaml	CWE-502 A08:2021-Software and Data Integrity Failures	app.py:81	semgrep
HIGH	Hardcoded JWT Secret / Weak Signing Key	CWE-798 A02:2021-Cryptographic Failures	app.py:13	semgrep
HIGH	Potential Server-Side Request Forgery (SSRF)	CWE-918 A10:2021-Server-Side Request Forgery	app.py:91	semgrep
MEDIUM	Overly Permissive CORS Configuration with Wildcard	CWE-942 A05:2021-Security Misconfiguration	src/userController.js:11	semgrep

3. STRIDE Threat Model Analysis

Category	Threat Title	Risk Level	Impact x Prob	Recommended Control
Tampering	Database Tampering & Manipulation via SQL Injection via Dynamic String Formatting / Concatenation	Critical	$5 \times 4 = 20$	Use Parameterized Queries / ORM Prepared Statements.
Information Disclosure	Credential Leakage / Information Disclosure (Hardcoded Secret: AWS Access Key ID)	Critical	$5 \times 4 = 20$	Store all secrets in environment variables or KMS / Vault secrets manager.
Information Disclosure	Credential Leakage / Information Disclosure (Hardcoded JWT Secret / Weak Signing Key)	Critical	$5 \times 4 = 20$	Store all secrets in environment variables or KMS / Vault secrets manager.
Spoofing	Unauthenticated API Request Spoofing / Missing Authentication	High	$4 \times 4 = 16$	Enforce JWT/OAuth2 Bearer token authentication middleware globally.
Elevation of Privilege	Broken Object Level Authorization (BOLA / IDOR)	High	$5 \times 3 = 15$	Validate user ownership on every database lookup before returning resource.
Denial of Service	Unbounded Payload Ingestion & Missing Rate Limiting	High	$3 \times 4 = 12$	Deploy IP/Token rate limiting and strict body size thresholds.
Repudiation	Insufficient Audit Logging for State-Changing Operations	Medium	$3 \times 3 = 9$	Implement centralized tamper-evident audit logging with user identity tracking.

4. Context-Aware Patch Recommendations

Patch for app.py • Status: PROPOSED • Re-scan Verified: Yes

Converted dynamic SQL string interpolation to parameterized query using bound parameter tuples (?), neutralizing SQL injection.

```
--- a/app.py
+++ b/app.py
@@ -45,10 +45,8 @@
conn = sqlite3.connect(DATABASE_PATH)
cursor = conn.cursor()

- # VULNERABLE: Direct SQL string interpolation allows ' OR '1'=1 injection
- query = f"SELECT id, username, role FROM users WHERE username = '{username}' AND password = '{password}'"
- cursor.execute(query)
- user = cursor.fetchone()
+
```

Patch for app.py • Status: PROPOSED • Re-scan Verified: Yes

Replaced shell=True and raw os.system calls with structured argument list subprocess execution to prevent command injection.

```

--- a/app.py
+++ b/app.py
@@ -66,9 +66,9 @@
@app.get("/api/tools/ping")
def ping_host(host: str):
# VULNERABLE: Executing shell command with unvalidated user input
- command = f"ping -c 1 {host}"
- status = os.system(command)
- return {"host": host, "status_code": status}
+ import subprocess
+ # Execute without shell=True to avoid command injection
+ status = subprocess.ru

```

Patch for src/userController.js • Status: PROPOSED • Re-scan Verified: No

Replaced hardcoded static secret with os.environ.get() dynamic runtime lookup.

```

--- a/src/userController.js
+++ b/src/userController.js
@@ -1,1 +1 @@
- // SECURITY ISSUE: Hardcoded AWS Access Key (CWE-798)
const AWS_ACCESS_KEY_ID = "AKIA*****MPLE";
const AWS_SECRET_ACCESS_KEY = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY";
+ [Remediated Code]

```